Secure TCP Service Exposure via WireGuard VPN

Technical Documentation

Project Update 1.1: Real-Time Infrastructure Monitoring

Embedded Systems & WireGuard Interface Health Tracking

Author: Jacek

February 21, 2026

Contents

1 Project Update Overview

The primary goal of this update (v1.1) is to implement a robust observability layer for the existing WireGuard infrastructure. Monitoring an embedded Linux device (Octagon SF4008) presents unique challenges, such as restricted library environments and the absence of standard package managers. This document details the deployment of a custom telemetry exporter designed to ensure high availability of the tunneled services.

2 Updated System Architecture

The architecture has been extended with a dedicated **Monitoring Node**. A critical design choice was made to poll the embedded device via its **Local IP Address**. This ensures diagnostic visibility even if the WireGuard tunnel itself fails.

- **Public VPS**: Acts as the public entry point and VPN concentrator.
- **Octagon SF4008 (Embedded)**: Initiates the encrypted tunnel to the VPS.
- **Monitoring VM (Debian 13)**: Centrally collects metrics via the local area network (LAN).

3 Implementation: Advanced WireGuard Exporter

Due to the lack of `systemd` and modern `glibc` support on the Enigma2 image, a custom Python-based metrics exporter was developed using the native `wg show dump` command.

3.1 Environment Preparation and Script Deployment

The configuration is performed via SSH. To ensure code integrity and correct directory structures, we use the *heredoc* method.

```
# 1. Create the native script directory for Enigma2
mkdir -p /usr/script

# 2. Deploy the Python exporter script using heredoc
cat << 'EOF' > /usr/script/wg_exporter.py
#!/usr/bin/python3
from http.server import BaseHTTPRequestHandler, HTTPServer
import subprocess

PORT = 9110
```

```
class MetricsHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == '/metrics':
            self.send_response(200)
            self.send_header('Content-Type', 'text/plain; version
               =0.0.4')
            self.end_headers()
            output = ""
            try:
                # Retrieve active WireGuard interfaces
                ifaces = subprocess.check_output(["wg", "show", "
                   interfaces"]).decode("utf-8").split()
                for iface in ifaces:
                    # Extract peer data using 'dump' for reliable
                       parsing
                    cmd = "wg show " + iface + " dump | tail -n +2"
                    wg_data = subprocess.check_output(cmd, shell=
                       True).decode("utf-8").strip()
                    for line in wg_data.split('\n'):
                        p = line.split('\t')
                        if len(p) >= 8:
                            peer = p[0]
                            try:
                                last_hs = int(p[4]); rx = int(p[5])
                                   ; tx = int(p[6])
                                output += f'
                                   wg_peer_last_handshake_seconds{{
                                   interface="{iface}",peer="{peer
                                   }"}} {last_hs}\n'
                                output += f'
                                   wg_peer_receive_bytes_total{{
                                   interface="{iface}",peer="{peer
                                   }"}} {rx}\n'
                                output += f'
                                   wg_peer_transmit_bytes_total{{
                                   interface="{iface}",peer="{peer
                                   }"}} {tx}\n'
                            except: continue
            except Exception as e:
                output = f"# Error: {str(e)}"
            self.wfile.write(output.encode('utf-8'))
        else:
            self.send_response(404); self.end_headers()

if __name__ == '__main__':
    server = HTTPServer(('0.0.0.0', PORT), MetricsHandler)
    server.serve_forever()
EOF
```

Listing 1: Deploying the Exporter via Terminal

3.2 Persistence and Execution

To ensure the monitoring service persists across reboots:

1. Assign execution privileges: `chmod +x /usr/script/wg`$_e$*xporter.pyInitializetheservice* : `/etc/init.d/wg-exporter restart`(*orappendtheexecutioncommandto*`rc.local`).

4 Prometheus Configuration

The Monitoring VM (Debian 13) is configured to scrape the embedded device using its **Local IP Address**.

```
scrape_configs:
  - job_name: 'octagon-sf4008-vpn'
    scrape_interval: 15s
    static_configs:
      - targets: ['XXX.XXX.XXX.XXX:9110'] # Target: Local IP
          Address
```

Listing 2: Prometheus Scrape Configuration

5 Visual Verification (Grafana)

The following visualizations confirm the successful ingestion of metrics. The dashboard provides real-time insights into tunnel health and throughput across multiple peers.

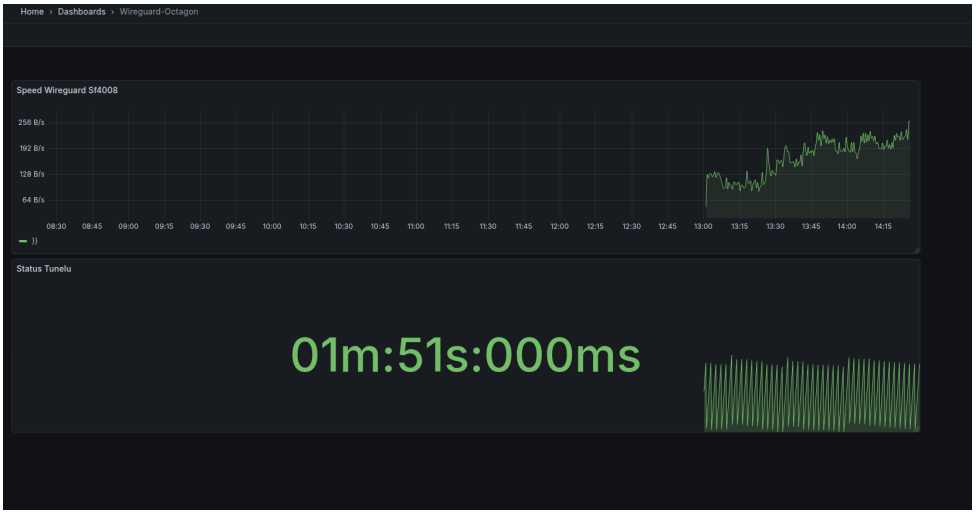

Figure 1: Handshake Age and Connectivity Status - Visual confirmation of active telemetry.

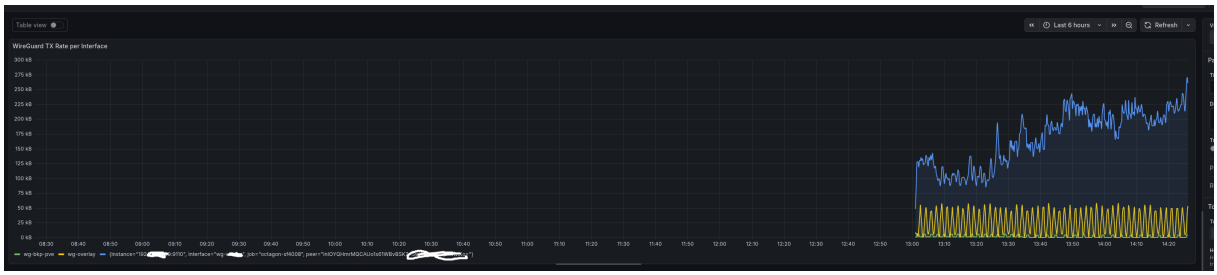

Figure 2: Comparative Performance Analysis of three WireGuard tunnels (Overlay, Backup, Octagon).

6 Technical Advantages

The implementation of a custom exporter using `wg show dump` provides several critical advantages:

- **Precision**: Using `dump` output eliminates parsing errors typically caused by dynamic peer IP headers or varying column widths in standard output.
- **Completeness**: Captures throughput (RX/TX) and the `last_handshake` parameter, enabling precise alerting for tunnel timeouts.
- **Standardization**: Utilizing the `/usr/script/` directory aligns with the native file structure of Enigma2 systems, ensuring maintainability.